

O'REILLY®

Agentic RAG with LangGraph

Sajal Sharma



About Me

- AI Engineer with 8+ years in ML, NLP & Software Engineering - across product and consulting
- Currently building AI-agents for various ventures at Liminal (prev. Menyala) - a venture studio in Singapore
- Created [Building AI Agents with LangGraph](#) on-demand course at O'Reilly
- Mentor at Udacity and University of Melbourne | Guest Lecturer at Yale
- Passionate about crafting composable agentic systems & sharing knowledge through sajalsharma.com

 [LinkedIn](#) | Email: contact@sajalsharma.com

Course schedule

- **Part 1: RAG with LangGraph**
 - RAG & LangGraph refresher
 - **Exercise:** Build a basic RAG workflow with LangGraph
- **Part 2: Agentic Patterns for RAG**
 - Agentic capabilities and mechanisms like query analysis, reflection, etc
 - **Exercise:** Single agent RAG with query routing
- **Part 3: Agentic RAG Architectures**
 - Composing patterns into full workflows
 - Corrective and adaptive RAG workflows
 - **Exercise:** Multi agent adaptive RAG pipeline



Part 1: RAG with LangGraph



RAG Refresher

Poll

What's your experience level with RAG?

- Never used
- Hobby project
- Pilot at work
- Production system at scale

RAG Recap

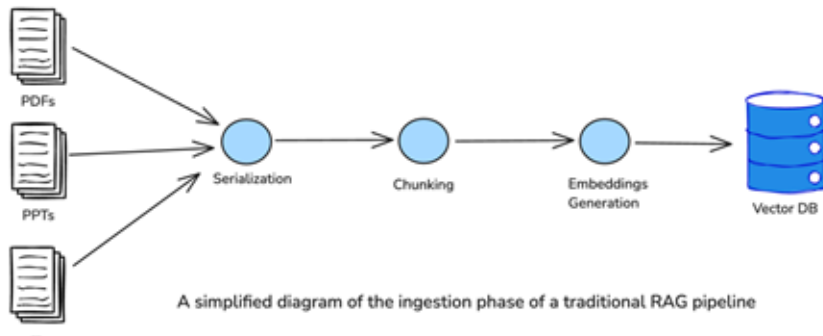
- **RAG (Retrieval-Augmented Generation)** enhances LLM outputs by retrieving relevant context before generating a response
- Bridges the gap between static model knowledge and dynamic, real-world information.
- The "R" in RAG
 - **Internal Sources:** Company documents, wikis, databases - anything in your system's data stores
 - **External Sources:** Public websites, APIs, news feeds, research papers, real-time data streams

Traditional RAG Workflow

Ingestion Phase

- **Key Ingestion Steps**

- Document Loading:** Plaintext, PDF, HTML, Markdown etc
- Serialisation** of images, tables, etc (converting to a text representation)
- Chunking:** Breaking documents into manageable pieces (200-1000 tokens)
- Embedding Generation:** Converting text chunks to high-dimensional vectors
- Storage:** Persisting vectors and metadata in specialised databases (usually vector dbs)

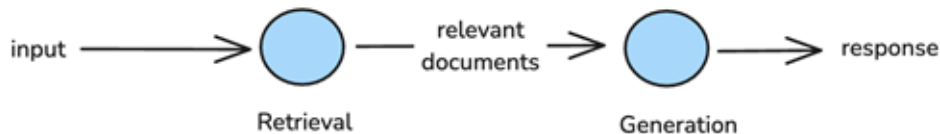


Traditional RAG Workflow

Inference Phase

- **Key Inference Steps**

- Query Processing & Enhancement:** Understanding user intent, extracting metadata filters, query expansion or decomposition, hypothetical document generation etc.
- Initial Retrieval:** Finding top-k candidate document chunks via vector/hybrid search
 - Optional: Re-ranking
- Generation:** Producing contextually grounded responses



A simplified diagram of the inference phase of a traditional RAG pipeline



Why RAG Continues to be Relevant

- **Foundation Model Limitations Persist**

- Knowledge Cutoffs: GPT-4o (Oct 2023), Claude 4 (March 2025), Gemini 2.5 Pro (January 2025)
- Context Constraints: Even 2M token windows have "lost in the middle" problem + high costs
- Hallucination Risks, raise liability concerns

- **Dynamic Knowledge Needs**

- Real-time Data: Financial markets, regulatory changes, academic research, coding (documentation updates)
- Enterprise Knowledge : Internal docs, policies, proprietary information

Why RAG Continues to be Relevant

- **Advantages compared to fine-tuning**
 - **Cost Effective:** No retraining, simpler infra, lower maintenance
 - **Flexible:** Real-time updates, granular control, version management
 - **Transparent:** Traceable sources, auditable



LangGraph Basics



Poll

Have you used LangGraph or any LLM orchestration frameworks before?

- Yes, LangGraph
- Yes, something else
- Heard of it, but never tried
- Never

LangGraph Basics

- **LangGraph Recap**
 - Framework for building **agentic workflows** with LLMs
 - **Graph-based**: Models workflows as directed graphs with cycles
 - **Stateful**: Maintains persistent state across execution steps
 - **Extras**: Built in persistence, streaming, human-in-the-loop capabilities

Nodes: The Building Blocks

- Python functions that process and transform state
- **Input:** Receive current state as parameter
- **Output:** Return state updates (partial or complete)
- **Flexibility:** Can contain any logic - LLM calls, tools, pure Python

```
def retrieve_docs(state: State):  
    """A node to retrieve relevant documents"""  
    query = state["question"]  
    documents = vector_store.similarity_search(query, k=5)  
    return {"context": documents} # Returns state updates
```

Edges: Defining Control Flow

- **Purpose:** Determine how execution flows between nodes.
- **Types:** Normal (direct), Conditional (dynamic), Entry/Exit points

```
# Normal edge - direct transition
workflow.add_edge("retrieve",
"generate")
# Conditional edge - dynamic routing
workflow.add_conditional_edges(
    "analyze_query",
    routing_function,
    {
        "simple": "direct_response",
        "complex": "retrieve_documents",
        "clarification": "ask_user"
    }
)
```

State Manager in LangGraph

- **Shared Context:** All nodes access the same state object
- **Persistent:** Maintained across execution steps
- **Typed:** Structured schema using TypedDict or Pydantic
- **Reducers:** Control how state updates are applied (overwrite, append, etc)

```
from typing_extensions import TypedDict

class RAGState(TypedDict):
    question: str # User query
    documents: list # Retrieved documents
    answer: str # Generated response
    metadata: dict # Additional
    information
```

Example: Graph Construction

```
from langgraph.graph import StateGraph, START, END

# 1. Step 1: Define state
class State(TypedDict):
    question: str
    context: list
    answer: str

# 2. Step 2: Create builder
builder = StateGraph(State)

# 3. Add nodes (retrieve_function and generate_function must
# be defined before this step)
builder.add_node("retrieve", retrieve_function)
builder.add_node("generate", generate_function)

# 4. Define edges
builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "generate")
builder.add_edge("generate", END)

# 5. Compile with options
graph = builder.compile()
```

Exercise: Creating a RAG Workflow using LangGraph



Q&A



Part 2: Agentic Patterns for RAG



Why go Agentic?

Limitations of Traditional RAG Systems

- **Fixed Pipeline**

- Traditional RAG follows a rigid sequence: Query → Retrieve → Generate
- Single round of retrieval from predefined sources
- No ability to refine search if first attempt fails
- System cannot adjust strategy based on query complexity

- **Source Selection Blindness**

- Cannot choose between multiple knowledge sources intelligently
- Treats all queries the same regardless of intent



Poll

Which limitation of RAG have you encountered most frequently?

- Poor document quality
- Fixed Pipeline
- Wrong source selection
- Something else (comment in chat)

What makes a system agentic?

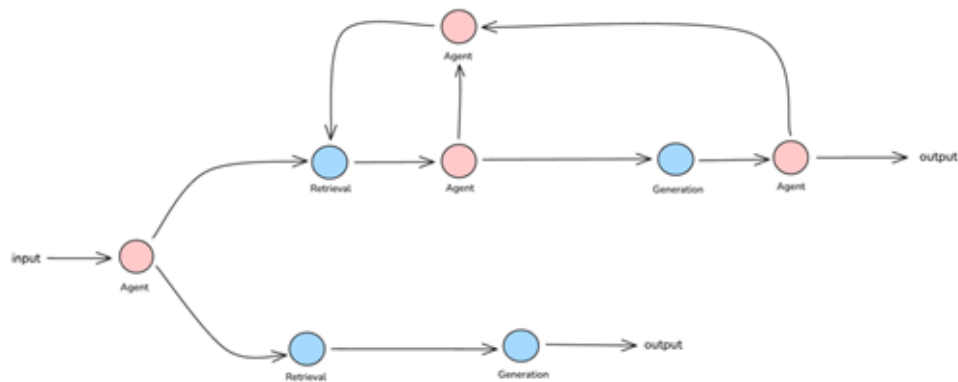
- Autonomy in decision-making - select best course of action
- Adaptive behaviour - adjusts strategy based on context
- Iterative improvement - refines approach through feedback

Agentic Capabilities for RAG

- **Query analysis:** Understands query nature and complexity to determine best course of action
- **Reasoning and Planning:** Ability to answer complex queries by breaking them down into multiple subqueries, and charting a course of action
- **Self-evaluation and reflection**
 - assesses retrieved documents to determine relevance
 - assess final outputs to determine out next best step

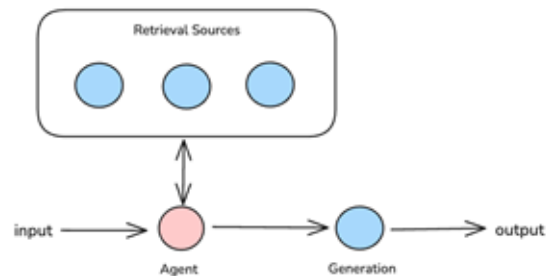
Flavours of Agentic RAG

Workflows



An Agentic RAG Workflow

Fully Autonomous



Fully Autonomous Agentic RAG



Agentic Patterns for RAG

Query Analysis

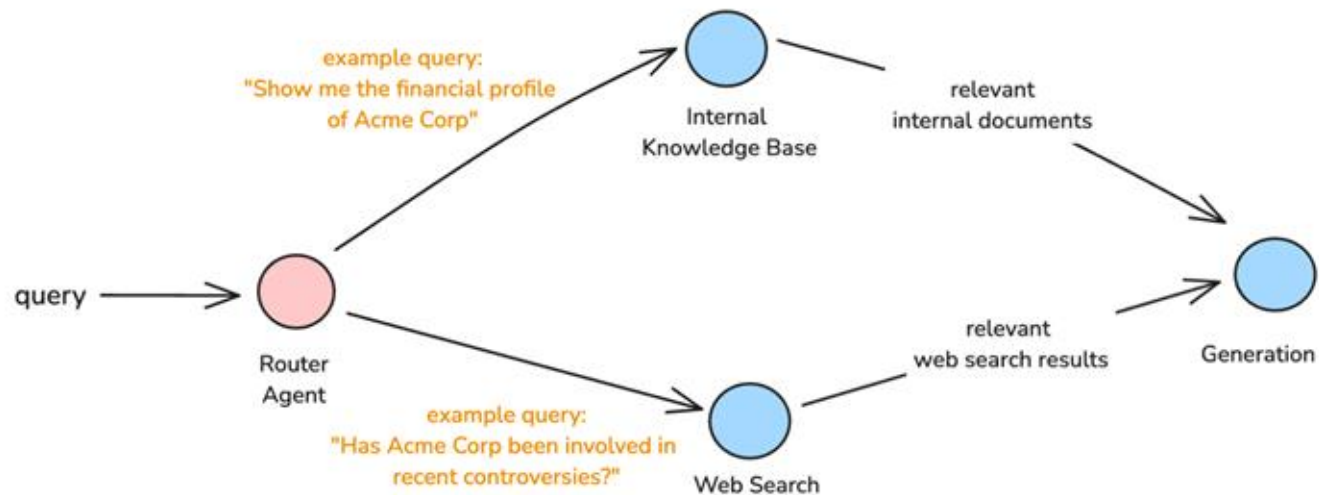
- **Purpose**

- **Understand query intent** before executing retrieval
- **Determine optimal approach** based on query characteristics
- **Extract relevant parameters** for targeted search

- **Decisions that can be made from analysis:**

- is retrieval needed at all?
- what's the most appropriate source?
- what retrieval strategy can be used? (for example: keyword or semantic or hybrid)
- what filters or constraints apply?

Query Routing



An example of an Agentic RAG workflow with a Router Agent

Query Routing

- **Single Source Routing**

- Classify query intent to select one optimal source
- Route to specific database or knowledge base
- Reduce noise from irrelevant sources

- **Multi Source Routing**

- Query multiple sources simultaneously
- Parallel retrieval for comprehensive coverage
- Merge and rank results from different sources

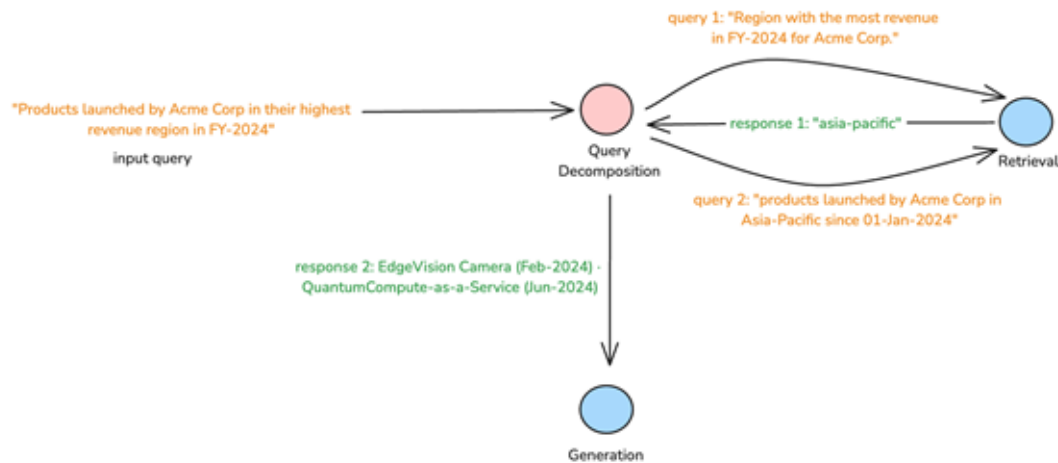
Query Decomposition

- **Purpose**
 - Break complex queries into sub-queries
 - Coordinate multiple knowledge sources for complete information
 - Identify information dependencies and create execution plan
- **Execution Strategies**
 - Sequential
 - Parallel

Query Decomposition: Execution Strategies

- **Sequential Execution**

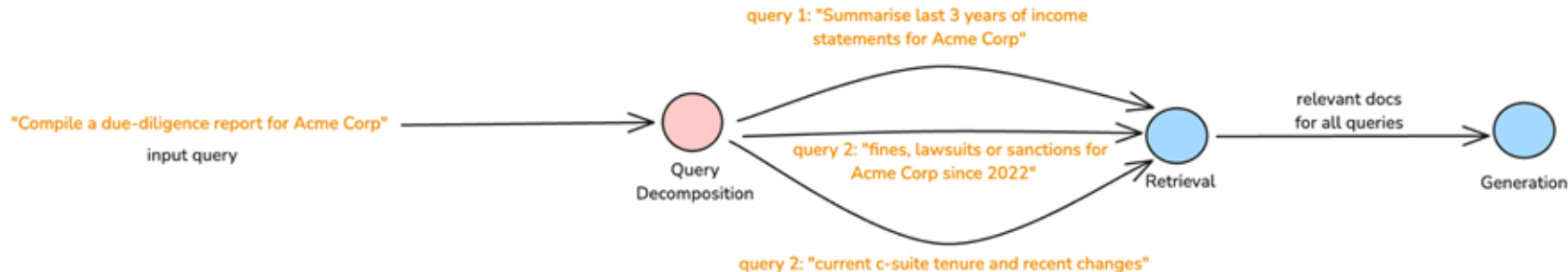
- use results from step 1 to inform step 2



An example of query decomposition with sequential execution

Query Decomposition: Execution Strategies

- **Parallel Execution**
 - Independent information gathering

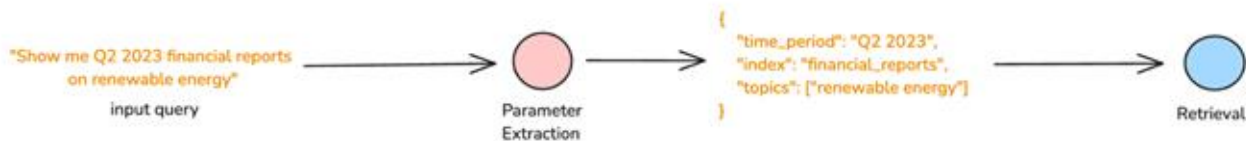


An example of query decomposition with parallel execution

Parameter Extraction

- **Purpose**

- Extract structured parameters from natural language queries
- Enable precise filtering in complex search systems, based on metadata of the ingested documents
- Improve retrieval accuracy through targeted constraints



An example of the extracting parameters from a query

Parameter Extraction: Examples

- **Time periods:** "Q2 2023", "last quarter", "2020-2022"
- **Categories:** "financial reports", "technical documentation"
- **Departments:** "HR", "Engineering", "Sales"
- **Product specifications:** "laptop", "wireless", "external monitor"



Break



Query Rewriting

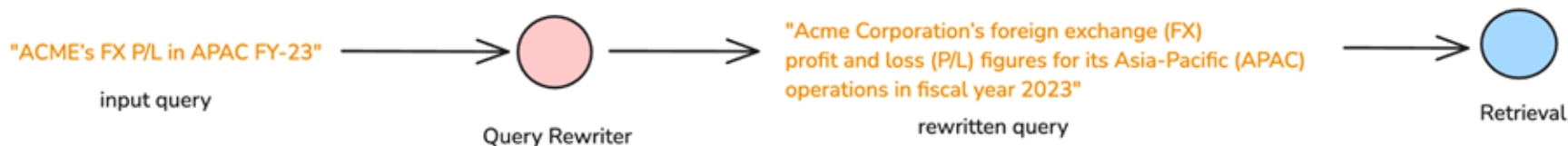
- Rewrite a query before searching
- **Purpose**
 - Rephrase vague queries
 - Add clarifying context in conversational systems
 - Bridge semantic gap between user language and document content

Query Rewriting: Techniques

- Expansion
- Contextualisation
- Reformulation
- Step-back prompting

Query Rewriting: Expansion

- add synonyms, spell out abbreviations, etc.. usually need to do this in specific domains or add business context.



An example of the query expansion pattern

Query Rewriting: Contextualisation

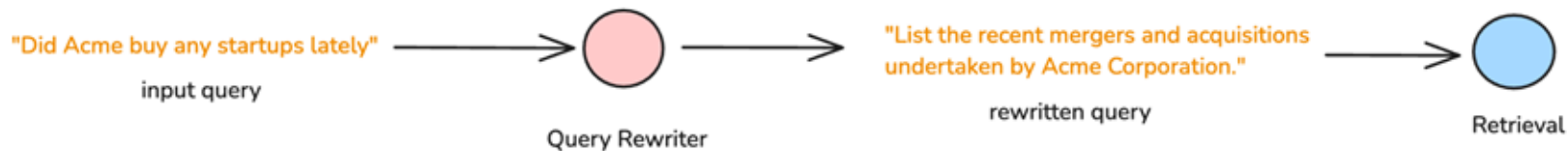
- Incorporate conversation history



An example of the query contextualisation pattern using conversation history

Query Rewriting: Reformulation

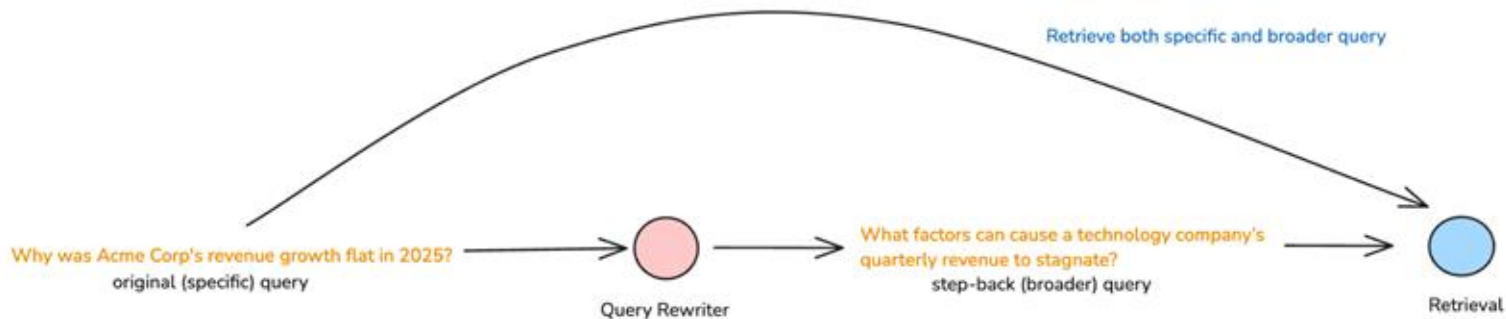
- Match document language patterns



An example of the query reformulation pattern

Query Rewriting: Step-back Prompting

- Abstract to higher level concepts
- **Benefit:** Retrieves both specific and general context



An example of the step back prompting pattern using conversation history

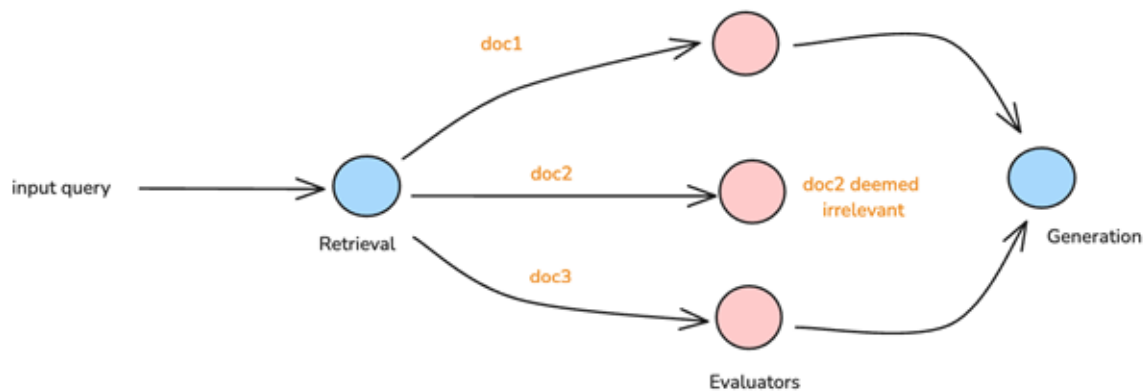
Reflection

- Reflection is an agentic mechanism that enables an AI agent to
 - evaluate its past actions or outputs
 - identify errors or gaps
 - adapt its future behavior to improve accuracy over time
- In Agentic RAG, reflection can be used to
 - Add a layer for filtering retrieved documents beyond relevance scores, by using LLM-as-a-judge
 - Assess outputs after the generation step, to identify next steps

Reflection: Capabilities

- **Retrieval quality assessment:** Grade document relevance
- **Response quality assessment:** Validate generated answers
- **Self-correction Loops:** Iterative improvement cycles when combined with other techniques

Reflection: Capabilities



An example of the self evaluation / reflection pattern

Reflection: Assessment Dimensions

- **Document Quality Dimensions**

- **Topical Relevance:** Does the document address the query topic?
- **Contextual Accuracy:** Is it relevant to the specific context? (e.g., right product, right problem)
- **Information Completeness:** Contains sufficient detail to answer the query?
- **Source credibility, temporal relevance**

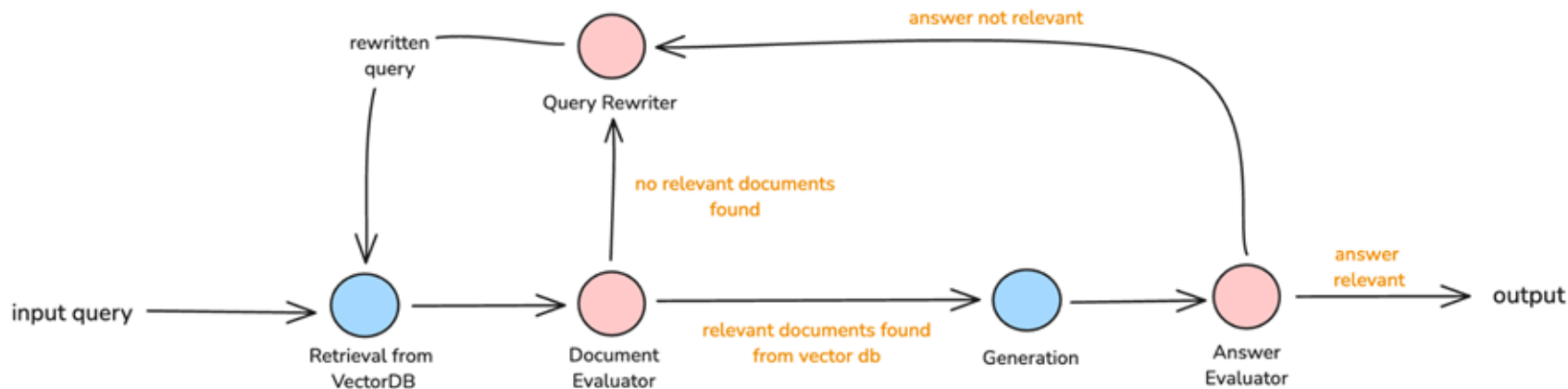
Reflection: Assessment Dimensions

- **Answer Quality Dimensions**
 - **Groundedness:** Are claims supported by the retrieved sources?
 - **Answer Relevance:** is the answer relevant to the query
 - **Completeness:** Does it fully address all aspects of the original query?
 - Frameworks like Ragas have baked in metrics

Adaptive Correction: Purpose

- Recover from poor retrieval or generation quality through iterative improvement
- Combine self-correction within strategies and fallback between strategies
- Ensure system reliability through graceful degradation

Adaptive Correction Patterns



An example of an adaptive correction patterns

Adaptive Correction: Termination Conditions

- Quality threshold achieved (success)
- Maximum iterations reached (effort limit)
- All fallback strategies exhausted (graceful failure: i don't know)

Adaptive Correction: Recommended Approach

Two Tier Correction

- **Tier 1 - Self-Correction Loops:**
 - Iterative refinement within same strategy
 - Refine query based on quality feedback
 - Adjust parameters or filters
 - Retry same knowledge source with improvements
- **Tier 2 - Fallback Mechanisms:**
 - Strategy switching when self-correction fails
 - Switch to alternative knowledge sources
 - Change retrieval methods (semantic → keyword → hybrid)
 - Escalate to external sources or human agents



Poll

Which agentic patterns have you tried in your RAG workflows?

- Query routing
- Query decomposition
- Reflection (docs / generated answers)
- Something else (comment in chat)

Example: Agentic RAG in a Customer Support System

- TechMart, an E-Commerce website for purchasing electronics has built a RAG based chatbot
- Data Sources
 - Product catalog database: specifications, features, compatibility
 - FAQ knowledge base: troubleshooting, how-to guides, common issues

Example: Agentic RAG in a Customer Support System

- **Failure 1: Wrong Source Routing**

- Customer: *"What's the battery life on the UltraBook Pro?"*
- Traditional RAG Routes to FAQ database → *"To extend battery life, close unused apps"*
- Should route to Product catalog → *"UltraBook Pro: 12-hour battery life"*
- 💡 **Agentic Solution:** Determine intent (specs vs. troubleshooting) and route to appropriate source

Example: Agentic RAG in a Customer Support System

- **Failure 2: Poor Document Quality (Semantic Similarity != Relevance)**
 - Customer: *"My wireless mouse keeps disconnecting"*
 - Retrieved: "Bluetooth headphone troubleshooting" (high similarity, wrong device)
 - Retrieved: "WiFi router disconnection issues" (high similarity, wrong problem)
 - 💡 **Agentic Solution:** Reflection mechanisms assess document relevance beyond similarity scores.

Example: Agentic RAG in a Customer Support System

- **Failure 3: Single-Hop Query Breakdown**

- Customer: *"Display issues with UltraBook Pro on external monitor. What ports should I plug in to and how do I fix flickering?"*
- Needs: Product specs (ports) + FAQ troubleshooting (flickering solutions)
- Traditional RAG: Single retrieval, incomplete response
- 💡 **Agentic Solution:** Query decomposition breaks complex query into sub-queries, coordinates multiple sources for complete answer

Choosing a Framework

Frameworks for Implementing Agentic RAG

- **LangGraph**

- Low level framework that excels at complex orchestration
- Graph-based architecture enables intricate decision trees and looping workflows
- Global state for the graph, though node to node state can also be defined

- **LlamaIndex**

- Data-centric framework optimised for rapid RAG development
- Event-driven patterns for workflow orchestration
 - Type annotation based routing to trigger events
 - step return based event dispatch (event activation)
- Events carry data between steps. State / data schema defined per event.

Recommendations for Choosing a Framework

- Principles matter more than the choice of framework. Pick the one you're already comfortable with - or that offers the most baked in functionality for your use case.
- Both offer pre-built patterns for implementing agentic workflows, including agentic rag
- You can use both if you want! Think of retrieval agent node using LlamaIndex if that suits your workflow.

Exercise: End-to-end Single Agent RAG



Q&A





Break

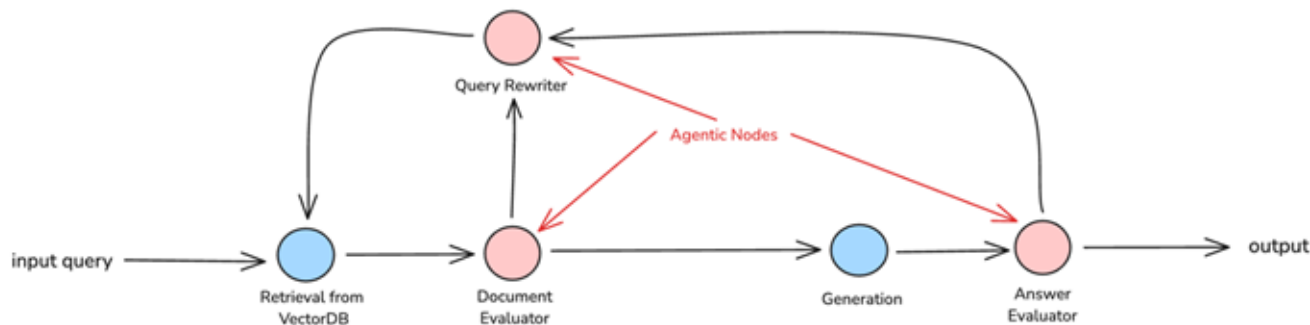
Part 3: Agentic RAG Architectures



Using LangGraph for Agentic RAG

Nodes as Agents

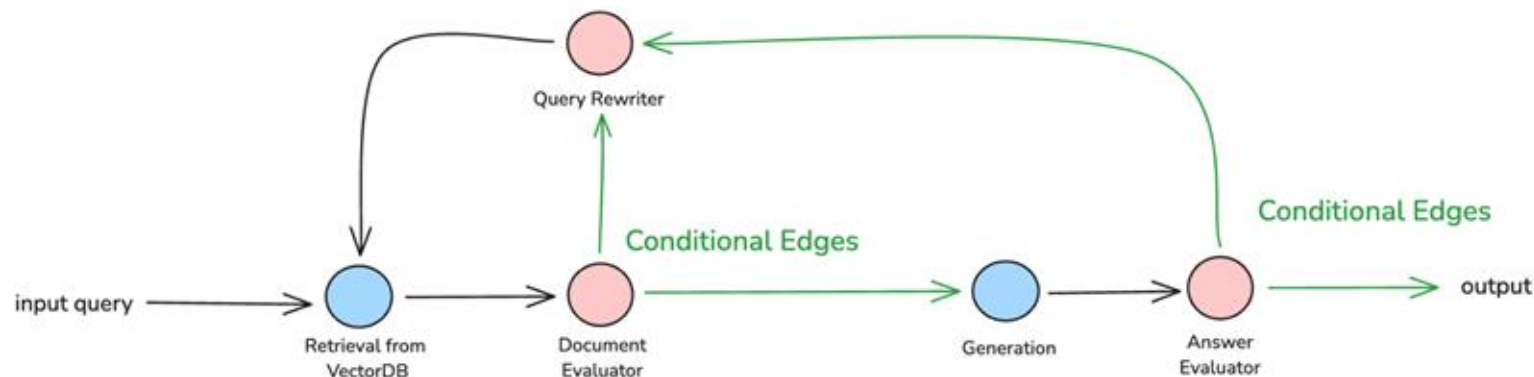
- Agentic logic can be implemented in individual nodes, example: query analysis node, retrieval node, reflection node, etc
- Nodes can update the state at the end of their operation



Agentic Nodes in a RAG Pipeline

Conditional Edges for Dynamic Flow Control

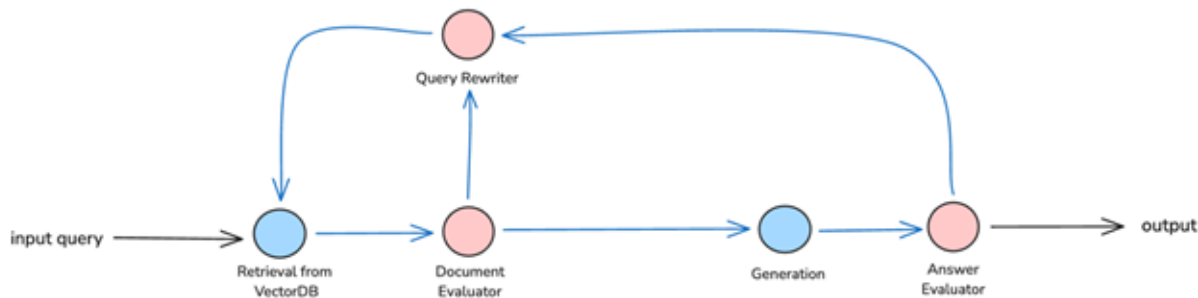
- Routing between different paths based on state evaluation
- Allows adapting to different scenarios



Conditional Edges in a RAG Pipeline

Self Correction through Cycles

- Iterative improvement via graph loops,
 - Example: retrieve -> assess -> refine -> retry



Cycles in a RAG Pipeline

Self Correction: Control Mechanisms

- **Iteration counter:** to track number of attempts
- **Quality threshold:** stop when quality sufficient
- **Time limits:** prevent excessive processing
- **State comparison:** stop when no improvements



Designing Agentic RAG Workflows

The Art of Composition: From Patterns to Workflows

- Lego-style thinking for working with patterns
 - Patterns are composable primitives - like Lego bricks
 - Real apps rarely map 1-1 into a single pattern
 - Design = choosing *which* bricks, *how many*, and *in what order*
- Conceptual process for designing your workflow
 - 1. Deconstruct user needs
 - Types of queries, what are good outputs
 - 2. Map patterns to sub-problems
 - 3. Sketch the flow / create your architecture

Map Patterns to Sub-Problems

- *Does the system need to understand user intent first?* → Start with **Query Analysis** and **Routing**.
- *Is the query complex and multi-faceted?* → Use **Planning** or **Query Decomposition** to break it down.
- *Are queries performed in a conversational context?* → Incorporate **Query Rewriting**.
- *Is retrieval quality a concern?* → Add a **Reflection** step to assess and filter retrieved documents.
- *Is the final answer accuracy critical?* → Use a second **Reflection** step to check for groundedness and completeness.
- *What happens if retrieval fails?* → Design an **Adaptive Correction** loop or fallback mechanism.

Sketch the Flow

- Connect the chosen patterns into a logical graph or workflow.
- You might find that for a simple Q&A bot, a **Route** → **Retrieve** → **Generate** → **Reflect** flow is enough.
- For a complex research agent, you might design a **Plan** → **Parallel Retrieval** → **Synthesize** → **Reflect** → **Generate** workflow.

Flow Execution Strategies

Sequential execution

- Tasks executed one after another, each step's output feeding the next
- Best suited for multi step reasoning, tasks requiring a strict order of operations, ex: multi hop queries that need intermediate info
- Inefficient for independent tasks / high latency

Parallelisation

- Multiple independent processes or subtasks are executed concurrently, synthesized in the end
- Best suited for multi source data aggregation, reflection of independently retrieved documents, etc
- Increased complexity in managing dependencies, may not be suited for all workflows



Sample Agentic RAG Workflows

Corrective RAG

- **Motivation**

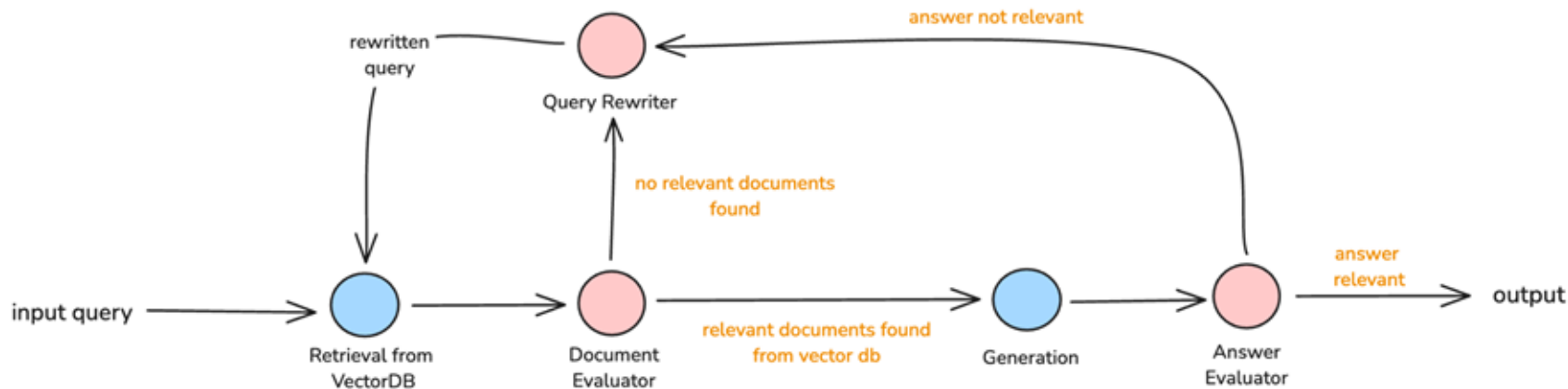
- Irrelevant document chunks retrieved in the retrieval step can distract LLMs, leading to incorrect or hallucinatory outputs
- Corrective RAG addresses this by self correcting retrieved knowledge before generation step

- **Architecture overview**

- Introduce a (usually lightweight) evaluation agent that assesses relevance of documents for a given query and assign a confidence score
- Determine whether to proceed with generation or take alternate course of action
 - the alternate course of action can be defined according to your needs

Corrective RAG

- We've seen this before: the adaptive correction pattern



An example of an adaptive correction patterns

Adaptive RAG

- **Motivation**

- Adaptive RAG is not a single technique but a design philosophy for building highly flexible pipelines
 - papers, tutorials etc will have some variation of the same philosophy

- **Core idea:**

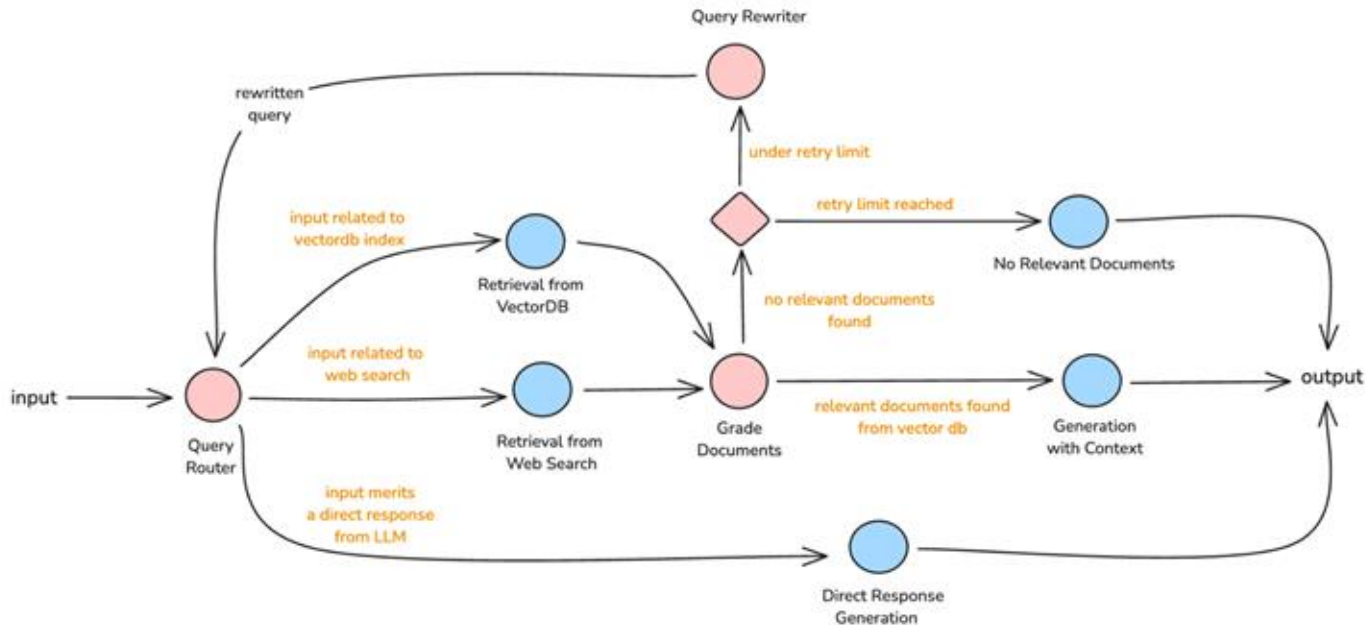
- single workflow cannot efficiently handle a vast spectrum of user queries
- compose multiple patterns into a dynamic system to select the most efficient path for each query

Adaptive RAG

- **Architecture overview**

- Usually has a query router at the entry point
- **Common paths**
 - Direct Response (no RAG)
 - RAG with optional Self Correction
 - RAG with Tool Use / External Sources
 - Fallback / retry mechanisms with query rewriting

Adaptive RAG



A visualisation of an adaptive rag workflow

Challenges and Mitigation Strategies

Latency and Performance Optimization

- **Challenge:** Multi-step workflows introduce significant latency
- **Mitigation Strategies:**
 - **Caching:** Implement caching for frequently queries results to avoid redundant computations
 - **Parallelization:** Where possible, execute independent workflow steps in parallel to improve throughput
 - **Model Optimization:** Use smaller language models (haiku, flash etc) for non-generative steps. Consider fine-tuning models (like phi mini, mistral small, SBERT) for query classification etc (research more)
 - **Efficient Search:** Where possible, optimise search algorithms if retrieval is slow
 - **Control LLM Output Lengths:** Use structured output and ask LLM to return only the necessary information (example: return a simple yes/no for document grading)

Challenges and Mitigation Strategies

Cost Management

- **Challenge:** Increased number of LLM calls can lead to significant operational costs
- **Mitigation Strategies:**
 - **Intelligent workflow design:** Provide paths to avoid complex operations, example: avoid retrieval + processing if no RAG needed for system
 - **Model Selection:** Choose the right model for the job, balancing capability, performance and cost. As mentioned previously, can use smaller models for routing / grading etc.
 - **Prompt Engineering:** Craft concise prompts and control output length to reduce token consumption.

Challenges and Mitigation Strategies

Debugging and Observability

- **Challenge:** Dynamic nature of agentic workflows can make debugging difficult
- **Mitigation Strategies:**
 - **Specialised tooling:** Use observability platforms like langsmith, langfuse (open source), to visualise and debug workflows
 - **Detailed tracing and logging:** Make sure the chosen tooling can capture the inputs, outputs and state at every step in the workflow.
 - **Real-time monitoring:** Set up alerts to notify of anomalies, performance degradation, or spikes in errors. They may also help capture user behaviour patterns.

Challenges and Mitigation Strategies

Reliability and Error Handling

- **Challenge:** Errors in a single step in the workflow can propagate throughout, leading to inaccurate final outputs
- **Mitigation Strategies:**
 - **Built in self correction:** A core tenet of Agentic-RAG, workflows should be designed to verify info, fall-back upon unexpected behaviour and fail gracefully.
 - **Confidence Scoring:** Allow mechanisms to flag low-confidence results for human review.

Challenges and Mitigation Strategies

Production Considerations

- **Challenge:** Moving from a prototype to a robust, scalable, and secure production environment requires careful planning across the entire AgentOps lifecycle.
- **Mitigation Strategies:**
 - **Workflow Design:** Design shallow workflows to minimise the number of hops.
 - **Eval-driven development:** Center your development process on a continuous cycle of evals. Have specific evals for every agentic step of your workflows.
 - Run automated eval jobs, for ex: regression suites (ex: weaviate rag eval) to guard against drift.
 - **Use state persistence:** Resume crashed runs rather than restart the entire chain
 - **Isolate orchestration, retrieval and inference:** scale each on the metric that reflects the bottleneck
 - **Scalable architecture:** Build for scale using modular components and cloud-native principles.

Exercise: Building a Multi Agent Adaptive RAG Pipeline



Discussion

What agentic mechanism will you try adding to your RAG pipeline?

Key Takeaways

- **RAG remains essential** for grounding LLMs in real-world, up-to-date information
- **Agentic workflows** bring adaptability, autonomy, and iteration to traditional RAG pipelines
- LangGraph is one of the frameworks that can be used to build Agentic RAG systems
- **Agentic patterns** like query analysis, routing, decomposition, reflection, and correction can be composed modularly
- **Balance performance and cost** using strategy-aware design, smaller models, and caching

Next Steps

-  **Experiment:** Apply patterns like corrective & adaptive RAG in your own projects
-  **Build:** Start with LangGraph templates and gradually compose your own agentic workflows
-  **Evaluate & Iterate:** Use eval-driven development and logging tools to refine your pipelines
- **Learn**
 -  **Read:** *AI Engineering* by Chip Huyen <https://learning.oreilly.com/library/view/ai-engineering/9781098166298/>
 -  **Take the Course:** *Building AI Agents with LangGraph (by me!)*
<https://learning.oreilly.com/course/building-ai-agents/0642572077884/>

Q&A



The O'Reilly logo is centered on a blue gradient background. It features the word "O'REILLY" in a white, bold, sans-serif font. A registered trademark symbol (®) is located at the top right of the "Y". In the background, there are faint, overlapping circular shapes in a darker shade of blue.

O'REILLY®